

FROST Guide

Threshold RedPallas: FROST and its Zcash integration surfaces

m@rek.onl

Abstract

Assuming Schnorr signatures from the *Crypto Guide*, RedPallas from the *Ironwood Guide*, PCZT from the *Wallet Guide*, and the finalizer model from the *Crosslink Guide*, this volume of *The Zcash Arboretum* documents threshold signing for Zcash. It constructs FROST as specified in RFC 9591, then its re-randomised form for Zcash spend authorisation and recoverability custody. The re-randomised Zcash stack is outside the completed cryptographic-library audits identified here; each design-stage claim is classified by the rigour of its source.

Contents

1	Scope, sources, and deployment surfaces	3
1.1	Scope	3
1.2	The source ladder	3
1.3	The three-bin rule	5
1.4	The three deployment surfaces	5
2	FROST	6
2.1	Shamir sharing and share conversion	7
2.2	Key generation: Pedersen’s DKG with a knowledge proof	7
2.3	Preprocessing and the two-round signing protocol	8
2.4	Why the binding factor exists: Drijvers and ROS	9
2.5	Nonce hygiene	10
2.6	The security theorem as stated	10
2.7	The corrected and extended analyses	11
2.8	RFC 9591 against the paper: what the RFC changed	12
3	Re-Randomized FROST for RedPallas	13
3.1	The mismatch: consensus verifies only randomised keys	13
3.2	Re-Randomized FROST: the construction	14
3.3	Unforgeability: TRUF, AOMDL, and Theorem 1	16
3.4	ZIP 312: the specification	18
3.5	The ciphersuite FROST(Pallas, BLAKE2b-512)	19
3.6	Published library implementation: <code>frost-rerandomized</code> and <code>reddsa</code>	20
3.7	Custody surfaces and classification	21
4	Deployment and open questions	22
4.1	The shipped library stack	22
4.2	The randomizer flow: draft ZIP against shipped crate	23
4.3	PCZT as the integration surface	24
4.4	Wallet-integration constraints	25
4.5	Transport: <code>frostd</code> and <code>frost-client</code>	25
4.6	Audit coverage, precisely	26
4.7	Custody of the ZIP-2005 quantum spending key	26
4.8	Crosslink finalizer keys	27
4.9	Classification and revisit triggers	27

1 Scope, sources, and deployment surfaces

This volume documents threshold custody of Zcash spend authority: how t of n parties jointly produce, without reconstructing the signing key during signing, a signature that consensus accepts as an ordinary RedPallas spend-authorisation signature. Three layers carry the construction. The base scheme is FROST, the two-round threshold Schnorr protocol of Komlo and Goldberg, specified in RFC 9591 (§2). The Zcash layer is Re-Randomized FROST (Gouvêa–Komlo, IACR ePrint 2024/436), which shifts every share by a common randomizer so that the aggregate signature verifies under the per-Action randomised key; ZIP 312 specifies it for Zcash, and the `frost-rerandomized` and `reddsa` crates ship it (§3). The deployment picture — the shipped stack, the proposed uses, and the gaps — closes the volume (§4).

Three custody surfaces anchor the treatment, one per rigour bin (§1.3): spend-authorisation custody for Orchard, where the construction is specified and shipped; custody of the ZIP-2005 quantum spending key `qsk`, where a Proposed consensus ZIP constrains a FROST ceremony whose recovery protocol it defers; and Crosslink finalizer keys, for which no threshold design was identified in the cited Crosslink corpus. The surfaces and their bins are fixed in §1.4.

1.1 Scope

The object under study is a t -of- n signing protocol whose output has the same format and verification rule as single-party spend authorisation. ZIP 312 states the requirements directly: the signatures must verify as Sapling or Orchard spend-authorisation signatures, and should meet the protocol specification’s “Signature with Re-Randomizable Keys” criteria. Plain FROST fails the first clause — it signs for a fixed group key, and consensus verifies only per-Action randomised keys — and closing that gap is the volume’s central construction (§3).

The volume develops RedPallas only, per its title. ZIP 312 itself defines two ciphersuites — FROST(Jubjub, BLAKE2b-512) for Sapling RedJubjub and FROST(Pallas, BLAKE2b-512) for Orchard RedPallas — and we cite the Jubjub suite where the ZIP fixes both at once, but develop only the Pallas suite (§3.5).

ZIP 312’s stated non-requirements bound the scope from below: it does not remove the Coordinator role; it defines no key-generation procedure (out of scope there, exactly as in RFC 9591); and it provides no network privacy. Key generation nevertheless appears twice in this volume, both times forced by sources: once as published with the base scheme (§2.2), and once as the unspecified-but-shipped DKG surface of the `reddsa` crate, binned in §1.3.

One altitude note governs every deployment claim. Availability is library-level, not consensus-level: no consensus rule names FROST, and none needed to change. The verifier accepts the aggregate because it is a valid RedPallas signature; as ZIP 312’s rationale puts it, the method by which the commitment R was generated is irrelevant to the verifier.

1.2 The source ladder

Table 1 pins the primary sources used for the full review. Repository heads were rechecked on 2026-09-05: the FROST-facing protocol text is unchanged at `zcash/zips e753a6a3`, and the algorithms and Zcash ciphersuite behaviour described below remain unchanged at `ZcashFoundation/frost 0966bd1` and `ZcashFoundation/reddsa 63c6ac0`. Local copies of the papers and RFC are archived with the volume’s drafts.

The RFC. RFC 9591, “The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures” (D. Connolly, Zcash Foundation; C. Komlo, University of Waterloo and Zcash Foundation; I. Goldberg, University of Waterloo; C. A. Wood, Cloudflare; IRTF CFRG stream, Category Informational, June 2024), records CFRG consensus on the base two-round protocol and leaves key generation out of scope. It explicitly says that it is not an Internet Standard and

Artefact	Pin / date	Evidence status
RFC 9591 (IRTF CFRG)	June 2024	Informational; CFRG consensus preprint; no venue
ePrint 2024/436 (Gouvêa-Komlo)	recv. 2024-03-13	Draft (Wallet category)
ZIP 312	6961098410, 2026-07-09	Proposed (Consensus category)
ZIP 2005	6961098410, 2026-07-09	shipped code; workspace 3.0.0
ZcashFoundation/frost	2016e44ba4, 2026-04-23	shipped code; crate 0.5.2
ZcashFoundation/reddsa	3792daa95e, 2026-05-22	demos; not for production
ZcashFoundation/frost-tools	06c0dbdb, 2026-08-12	peer-reviewed antecedents
ePrint 2020/852; 2022/833	SAC 2020; CRYPTO 2022	

Table 1: Primary FROST sources used for the full review. Current heads rechecked 2026-09-05.

that IRTF results might not be suitable for deployment. Two of its four authors are Zcash Foundation, so the RFC and Zcash implementation share provenance. Its deltas against the SAC 2020 paper are catalogued in §2.8.

The paper. “Re-Randomized FROST”, by Gouvêa and Komlo (both Zcash Foundation), is IACR ePrint 2024/436 (<https://eprint.iacr.org/2024/436>): received 2024-03-13, approved 2024-03-15, publication info “Preprint”, no peer-reviewed venue as of 2026-09-05. We read it in full and work from it throughout §3. Its headline Theorem 1 claims unforgeability in the random-oracle model under AOMDL. That game restricts oracle queries to declared algebraic combinations, unlike the OMDL game used for plain FROST in BCKMTZ. The printed reduction has material gaps documented in §3.3; this volume therefore does not treat that claim as an established security theorem.

The ZIP. ZIP 312, “FROST for Spend Authorization Multisignatures” (owners Conrado Gouvêa, Chelsea Komlo, and Deirdre Connolly; Status Draft, Category Wallet; Discussions-To zcash/zips#382, pull request zcash/zips#662), is the Zcash specification of the construction.¹ It is visibly unfinished — its Created field still reads 2022-08-dd, day unfilled — so we cite it as a moving object, by repository commit (zcash/zips @ 6961098410, 2026-07-09) and Draft status, never as a stable specification. Its threat model trusts the Coordinator and every key-share holder with transaction privacy and unlinkability, but its security requirement says that a rogue Coordinator must not be able to create a signed transaction without the approval of MIN_PARTICIPANTS participants. What it specifies — the unlinkability requirement, the randomizer flow, and the two ciphersuites — is developed in §3.4 and §3.5.

The crates. Repository ZcashFoundation/frost (@ 2016e44ba4, 2026-04-23) is a Rust workspace at version 3.0.0 (rust-version 1.81): crate frost-core carries the ciphersuite-generic protocol; five ciphersuite crates instantiate the RFC (ed25519, ed448, P-256, ristretto255, and secp256k1), while the separate frost-secp256k1-tr crate implements a Taproot-oriented suite; frost-rerandomized carries the re-randomised layer, and gencode generates the per-suite code. The repository also vendors an audit report dated 2021-03-23 (zcash-frost-audit-report-20210323.pdf) and the source of the ZF FROST book (published at frost.zfnd.org), whose “FROST with Zcash” chapters (zcash-devtool demo, Ywallet demo, FROST server) and serialisation-format page — the page ZIP 312 cites for encode_signing_package — feed §4. Repository ZcashFoundation/reddsa (@ 3792daa95e, 2026-05-22; crate version 0.5.2) holds the Zcash ciphersuites: its frost feature enables frost-rerandomized 3.0.0, and src/frost/redpallas.rs defines FROST(Pallas, BLAKE2b-512) as struct PallasBlake2b512, ZIP 312’s reference implementation surface (§3.6). Crate reddsa also ships machinery ZIP 312 does not specify — DKG hash personalisations and even-Y normalisation hooks — binned in §1.3. Its README marks the frost

¹The FROST ZIP is ZIP 312 alone. ZIP 313, its numeric neighbor, is “Reduce Conventional Transaction Fee to 1000 satoshis” — Status Obsolete, obsoleted by ZIP 317 — and has nothing to do with FROST.

feature experimental while ZIP 312 remains Draft and explicitly excludes that feature from the crate’s SemVer and MSRV guarantees.

The antecedent papers. FROST itself is Komlo–Goldberg, IACR ePrint 2020/852 (SAC 2020); the security analysis the re-randomised extraction argument extends is Bellare–Crites–Komlo–Maller–Tessaro–Zhu, “Better than Advertised Security for Non-Interactive Threshold Signatures” (CRYPTO 2022; ePrint 2022/833). They are the paper’s references [10] and [1] respectively; §2 works from both.

1.3 The three-bin rule

The series rule is binary for implementation behaviour and ternary for design. A claim about implementation behaviour cites the implementation — crate and file — and the specification section it realises; a claim about a design-stage protocol is classified *specified*, *designed but unspecified*, or an *open problem*, and the bin stays visible in the prose. This volume sits astride the boundary: one surface is shipped code, one is a consensus proposal, one is an acknowledged gap. Applied once, here, for the whole volume:

1. Re-Randomized FROST for spend-authorisation custody is *specified*: RFC 9591 published June 2024, ZIP 312 in Draft, ePrint 2024/436, and shipped code — `frost-rerandomized` 3.0.0 and `reddsa` 0.5.2. The latter’s FROST feature is explicitly experimental. Availability is library-level, not consensus-level (§1.1).
2. Custody of the ZIP-2005 quantum spending key is specified at the ZIP level (Status Proposed, Category Consensus), but the Recovery Protocol it exists to serve is “to be introduced, potentially, in a future upgrade” — *designed but unspecified*.
3. Crosslink finalizer threshold custody is an *open problem*: the *Crosslink Guide*’s own classification, which we adopt.

One surface refuses the clean split, and we bin it here once. No ZIP specifies FROST key generation: ZIP 312 declares it out of scope (as does RFC 9591), and ZIP 271’s May-2025 note confirms the gap — ZIP 312’s missing key-generation text is its stated reason not to couple disbursement of the *lock-box* (the in-protocol reserve that has accrued a portion of each block subsidy since NU6, per ZIP 2001) to that specification. Yet crate `reddsa` ships DKG support: hash personalisations `FROST_RedPallasD` (HDKG) and `FROST_RedPallasI` (HID), beyond ZIP 312’s H1–H5 and HR, plus even-*Y* normalisation hooks (`post_generate/post_dkg`, applying `into_even_y` to secret shares and the public key package) so that the group verifying key meets the encoding constraint of Orchard’s spend validating key. Deployed code with no specification above it: we classify the DKG surface *designed but unspecified* and cite the crate at commit 3792daa95e wherever we rely on it.

1.4 The three deployment surfaces

Spend-authorisation custody (specified). The product ZIP 312 promises is signatures “indistinguishable from RedPallas Orchard Spend Authorization Signatures”. The single-party object being imitated is settled in the *Ironwood Guide*: §“Spend authorisation: RedPallas re-randomisation” constructs $rsk := ask + \alpha$ and $rk := ak + [\alpha] G^{\text{SpendAuth}} = [rsk] G^{\text{SpendAuth}}$, with `GenRandom` hashing 80 uniform bytes through H° , and §“RedPallas unforgeability” proves EUF-CMA from discrete log in the random-oracle model. Per the layering rule we cite both and re-prove neither; this volume’s work begins where the single signer ends, at splitting `ask` into shares that a signing coalition need not reassemble (§3). The wallet-side integration path is likewise on the record: the *Wallet Guide*, §“Partially created transactions (PCZT)”, records ZIP 374’s motivation (iii), threshold signing (for example FROST, RFC 9591) — the multiparty transaction format FROST signing rides in §4.3.

Custody of the quantum spending key (designed but unspecified). ZIP 2005, “Ironwood Quantum Recoverability” (owners Daira-Emma Hopwood and Jack Grigg; Status Proposed, Category Consensus; created 2025-03-31), writes FROST into its requirements twice: the design should be fully compatible with FROST threshold multisignatures (citing ZIP 312), and should lose no pre-quantum security when FROST and/or hardware wallets are used. Its “Usage with FROST” section then constrains the ceremony: the point ak is derived jointly, via the FROST DKG, from shares of ask ; the participants must privately agree on a value sk and use it with $use_qsk = \text{true}$ to derive nk , qsk , qk , and $rivk_ext = H_{qk}^{rivk_ext}(\text{Extract}_p(ak), nk)$; all participants must obtain the same extracted key $\text{Extract}_p(ak)$, nk , and $rivk_ext$. Subject to the specified derivations, the ZIP says that this also implies equal qsk and qk under collision resistance of H^{qk} and H^{rivk_ext} . Each participant must hold qsk at least as securely as ask . The outlined Recovery Protocol would require a RedDSA signature under ak — preserving t -of- n for as long as discrete log on Pallas stands — together with a proof SoK^{qsk} of knowledge of a qsk with $H^{qk}(qsk) = qk$. The asymmetry is the point to retain: the key qsk is held by every participant, so a quantum adversary with qsk and the ability to solve discrete logarithms on Pallas no longer faces a threshold; the ZIP accordingly recommends moving funds to a fully post-quantum threshold protocol once one is available. The deferred Recovery Protocol is what parks this surface in the middle bin; the full treatment is §4.7.

Crosslink finalizer keys (open problem). The *Crosslink Guide*, §“Finalizer keys”, records the prototype state: one ed25519 key per finalizer (ed25519-zebra), a 76-byte vote format signed by appending a 64-byte ed25519 signature over those 76 bytes, and — by negative search over the four Crosslink repositories (tfl-book, crosslink-protocol-guide, crosslink-spec, zebra-crosslink), dated 2026-07-15 there — no FROST or threshold-signing design for finalizer keys sourced anywhere in the corpus. That guide’s classification bins threshold custody, key rotation, revocation, and initial-roster formation as open problems, and observes that FROST’s presence in the PCZT flow makes the absence “a visible gap rather than an exotic ask”. We adopt the classification and return to the surface in §4.8.

Adjacent surfaces, cited but not developed. Three further ZIPs touch FROST and mark how wide the surface is growing. ZIP 271 (“Dev Fund Extension and One-Time Disbursement”, Proposed) anticipates spending lockbox one-time-disbursement chunks “to a FROST address”, and its May-2025 note that ZIP 312 does not specify key generation is its stated reason not to couple the two specifications. ZIP 326 (“NU6.3 Consequences for Wallets”, Draft, created 2026-06-30) cites ZIP 2005’s “Usage with FROST” section. ZIP 374 (“Partially Created Zcash Transaction Format”, Draft, created 2024-12-09) lists FROST per RFC 9591 under its multiparty-transactions motivation. Each appears in this volume exactly where it bears (§4); none is developed as a surface of its own.

2 FROST

The base object of this volume is the threshold Schnorr scheme FROST (*Flexible Round-Optimized Schnorr Threshold* signatures) of Komlo and Goldberg, published at SAC 2020 and maintained as IACR ePrint 2020/852; we work from the final revision of 22 December 2020. The scheme lets any t of n parties, each holding a Shamir share of a signing key s , produce a signature (R, z) that verifies under the ordinary single-party Schnorr equation for the group public key $Y = g^s$. The signature carries no threshold marker, and a verifier applies the same equation as it would to a single-signer signature. That compatibility is what lets the scheme use Zcash’s existing RedDSA validators: the consensus rules do not observe the threshold ceremony. The deployment surfaces this volume targets are spend-authorisation custody over RedPallas (the *Ironwood Guide*, §“Spend authorisation: RedPallas re-randomisation”), custody of the ZIP-2005 quantum spending key (whose “Usage with FROST” section constrains the key generation), and Crosslink finalizer keys (the *Crosslink Guide*, §“Finalizer keys”).

Throughout, $\mathbb{G}$ is a group of prime order q with generator g , and H, H_1, H_2 are hash functions with outputs in $\mathbb{Z}_q^*$, modelled as random oracles. The paper fixes the single-party scheme it must match — the Schnorr signature scheme of the *Crypto Guide*, §“From identification to signature via the Fiat-Shamir transform” — restated here in the paper’s multiplicative notation: a Schnorr signature over m under secret key $s \in \mathbb{Z}_q$ and public key $Y = g^s \in \mathbb{G}$ is produced by sampling a nonce $k \xleftarrow{\$} \mathbb{Z}_q$, setting $R = g^k$, $c = H(R, Y, m)$, $z = k + s \cdot c$, and outputting $\sigma = (R, z)$; verification recomputes c and accepts iff $R = g^z \cdot Y^{-c}$. FROST’s whole design problem is to compute the pair (R, z) jointly, without reconstructing the nonce or signing key during signing and without opening the concurrency attacks described in §2.4.

2.1 Shamir sharing and share conversion

Two secret-sharing mechanisms cooperate in FROST, and keeping them distinct is the key to reading the signing equation.

Shamir sharing (the long-lived key). A dealer — or, in §2.2, every participant acting as a dealer simultaneously — selects $t-1$ random coefficients $a_1, \dots, a_{t-1}$ and forms the degree- $(t-1)$ polynomial

$$f(x) = s + \sum_{i=1}^{t-1} a_i x^i, \quad f(0) = s,$$

giving participant P_i the share $(i, f(i))$. Any t shares determine f by Lagrange interpolation and hence $s = f(0)$; fewer than t reveal nothing. For a signing set $S = \{p_1, \dots, p_\alpha\}$ of α participant identifiers ($t \leq \alpha \leq n$), the interpolation weight of participant i at zero is the Lagrange coefficient

$$\lambda_i = \prod_{j=1, j \neq i}^{\alpha} \frac{p_j}{p_j - p_i},$$

computable from the identifiers alone — the coalition can be chosen *after* shares are dealt, which is what makes FROST’s “any t of n , decided at signing time” flexibility possible.

Additive sharing and conversion (the per-signature nonce). A sum $s = \sum_i s_i$ with each summand chosen locally is an additive sharing: it needs no dealer and no interaction. The Cramer–Damgård–Ishai share conversion observation, used by FROST in its simplest case, is that additive shares convert locally to Shamir form: if $s = \sum_{i \in S} s_i$, then the values s_i/λ_i are Shamir shares of the same s (evaluations of a degree- $(\alpha-1)$ polynomial interpolating them). FROST uses exactly this to make the group nonce k : each signer contributes an additively-shared summand non-interactively, and the λ_i weights in the signing equation perform the conversion so that the response aggregates by plain summation. No DKG runs at signing time.

2.2 Key generation: Pedersen’s DKG with a knowledge proof

FROST generates its long-lived shares with a variant of Pedersen’s DKG — n parallel executions of Feldman’s verifiable secret sharing, one with each participant as dealer — hardened with a proof of knowledge of each dealer’s free coefficient. The paper’s Figure 1, reproduced:

Round 1.

1. Every participant P_i samples t random values $(a_{i0}, \dots, a_{i(t-1)}) \xleftarrow{\$} \mathbb{Z}_q$ and defines the degree- $(t-1)$ polynomial $f_i(x) = \sum_{j=0}^{t-1} a_{ij} x^j$.

2. Every P_i computes a proof of knowledge of a_{i0} : with $k \xleftarrow{\$} \mathbb{Z}_q$, $R_i = g^k$, $c_i = H(i, \Phi, g^{a_{i0}}, R_i)$, $\mu_i = k + a_{i0} \cdot c_i$, set $\sigma_i = (R_i, \mu_i)$, where Φ is a context string preventing replay across ceremonies.
3. Every P_i computes the public commitment $\vec{C}_i = \langle \phi_{i0}, \dots, \phi_{i(t-1)} \rangle$ with $\phi_{ij} = g^{a_{ij}}$.
4. Every P_i broadcasts $\vec{C}_i, \sigma_i$.
5. On receiving $\vec{C}_\ell, \sigma_\ell$, participant P_i verifies $R_\ell \stackrel{?}{=} g^{\mu_\ell} \cdot \phi_{\ell 0}^{-c_\ell}$ with $c_\ell = H(\ell, \Phi, \phi_{\ell 0}, R_\ell)$, *aborting on failure*; on success all σ_ℓ are deleted.

Round 2.

1. Each P_i securely sends each other participant P_ℓ the share $(\ell, f_i(\ell))$, then deletes f_i and every share except its own $(i, f_i(i))$.
2. Each P_i verifies each received share against the dealer's commitment:

$$g^{f_\ell(i)} \stackrel{?}{=} \prod_{k=0}^{t-1} \phi_{\ell k}^{i^k \bmod q},$$

aborting if the check fails.

3. Each P_i computes its long-lived signing share $s_i = \sum_{\ell=1}^n f_\ell(i)$, stores it securely, and deletes the $f_\ell(i)$.
4. Each P_i computes its public verification share $Y_i = g^{s_i}$ and the group public key $Y = \prod_{j=1}^n \phi_{j0}$.

Three design decisions deserve comment. First, the *proof of knowledge of a_{i0}* is the delta over plain Pedersen DKG: it addresses rogue-key attacks in the $t \geq n/2$ regime, where a participant could otherwise choose its commitment as a function of others' to bias the aggregate key (the fix was suggested to the authors by Shlomovits and Turkel; the paper's changelog dates it to the 2020-07-08 revision). Second, the *complaint handling* is abort-only: where Gennaro et al. add a complaint/disqualification phase to make the DKG robust, FROST aborts on any failed share check and expects the cause to be investigated out of band; the paper notes that implementations wanting robustness can substitute the Gennaro et al. construction. Third, a *consistent view* of the broadcast commitments $\vec{C}_i$ across participants is assumed, not provided — the paper defers to whatever broadcast mechanism the deployment supplies. Each participant ends holding (i, s_i) , with $Y_i = g^{s_i}$ public for share verification during signing and Y the single key the outside world sees.

2.3 Preprocessing and the two-round signing protocol

Signing splits into a message-independent *preprocess* stage and a single online round (equivalently, run as one two-round protocol with the commitment exchange inlined). A semi-trusted *signature aggregator* SA coordinates. The paper trusts it for publication and correct misbehaviour reporting, not for unforgeability, and claims in its security model that a malicious SA can deny service or falsely accuse but cannot learn the private key or cause an unapproved message to be signed.

Preprocess(π) (paper Figure 2). Each participant P_i generates π single-use nonce pairs and publishes their commitments: for $1 \leq j \leq \pi$,

$$(d_{ij}, e_{ij}) \xleftarrow{\$} \mathbb{Z}_q^* \times \mathbb{Z}_q^*, \quad (D_{ij}, E_{ij}) = (g^{d_{ij}}, g^{e_{ij}}),$$

appending (D_{ij}, E_{ij}) to a list L_i published to a predetermined location, and storing the secret halves for later.

Sign(m) (paper Figure 3). The aggregator SA selects a signing set S of α participants ($t \leq \alpha \leq n$), fetches each member's next unused commitment pair, and forms the ordered list $B = \langle (i, D_i, E_i) \rangle_{i \in S}$, sending (m, B) to every $P_i, i \in S$. Each participant then:

1. validates m (a signer signs only messages it is willing to sign) and checks $D_\ell, E_\ell \in \mathbb{G}^*$ for every commitment in B , aborting on failure;
2. computes the *binding values*

$$\rho_\ell = H_1(\ell, m, B), \quad \ell \in S,$$

the group commitment and challenge

$$R = \prod_{\ell \in S} D_\ell \cdot (E_\ell)^{\rho_\ell}, \quad c = H_2(R, Y, m);$$

3. computes its response, converting its Shamir share to the additive aggregation domain via λ_i (over the set S):

$$z_i = d_i + (e_i \cdot \rho_i) + \lambda_i \cdot s_i \cdot c;$$

4. securely deletes $((d_i, D_i), (e_i, E_i))$, then returns z_i to SA.

Finally SA recomputes $\rho_i, R_i = D_i \cdot (E_i)^{\rho_i}, R = \prod_{i \in S} R_i$ and c , and verifies every share against the public data:

$$g^{z_i} \stackrel{?}{=} R_i \cdot Y_i^{c \cdot \lambda_i},$$

identifying and reporting the misbehaving participant and aborting if any check fails; otherwise it publishes $\sigma = (R, z)$ with $z = \sum_{i \in S} z_i$.

Correctness. The proof is two interpolations (paper §6.1). The key polynomial $F_1(x) = \sum_{j=1}^n f_j(x)$ from key generation has $s_i = F_1(i)$ and $s = F_1(0)$. The nonce values $(i, (d_i + e_i \cdot \rho_i)/\lambda_i)$ define a degree- $(\alpha - 1)$ polynomial $F_2(x)$ with $F_2(0) = \sum_{i \in S} (d_i + e_i \cdot \rho_i) = k$. Setting $F_3(x) = F_2(x) + c \cdot F_1(x)$, each response is $z_i = \lambda_i F_3(i)$, so $z = \sum_{i \in S} z_i$ interpolates $F_3(0) = k + c \cdot s$; with $R = g^k$ and $c = H_2(R, Y, m)$, the pair (R, z) is a correct Schnorr signature on m .

2.4 Why the binding factor exists: Drijvers and ROS

The one non-obvious ingredient is $\rho_i = H_1(i, m, B)$. The binding factor is FROST's answer to concurrency attacks that affected or constrained the earlier two-round schemes discussed by the paper.

The Drijvers attack. Against two-round Schnorr multisignatures in which the adversary may open T parallel sessions, Drijvers et al. showed how to forge by finding a hash output that is a sum of others: $c^* = H(R^*, Y, m^*)$ with $c^* = \sum_{j=1}^T H(R_j, Y, m_j)$, using Wagner's k -tree algorithm for the generalised birthday problem in time $O(\kappa \cdot b \cdot 2^{b/(1+\lg \kappa)})$ for $\kappa = T + 1$ and b the bitlength of the group order. The attack transfers from the n -of- n multisignature setting to a t -of- n threshold scheme with an adversary controlling up to $t - 1$ signers, and it is why the Gennaro et al. scheme must cap its signing parallelism. Benhamouda et al.'s subsequent polynomial-time ROS solver (for $\ell > \lg q$ parallel sessions) turned the same structural weakness from subexponential to practical against schemes including two-round MuSig and GJKR-DKG-based threshold Schnorr; their paper concludes that “the current version of” FROST is *not* affected.

How binding addresses it. Because each signer derives $\rho_i = H_1(i, m, B)$ before responding, its share z_i is bound to this message, this signing set, and this exact commitment list. Except with random-oracle collisions, changing those inputs alters some ρ_ℓ , hence R and c , so directly transplanting a response fails verification. The paper’s own generalisation (its Equation 1, with linear combinations γ_j) states the residual attack surface precisely: the forger must find

$$H_2(R^*, Y, m^*) = \sum_j \gamma_j \cdot H_2(R_j, Y, m_j),$$

where $R^* = g^{r^*} \cdot \prod_j \hat{R}_j^{\gamma_j}$ is itself a function of every input on the right-hand side. Wagner’s algorithm needs the two sides of the collision to vary independently; here every candidate combination on the right moves the left-hand target, degrading the generalised birthday collision into multiple preimage attacks. The paper’s footnote 7 concedes this step is heuristic: sufficiency of the condition is immediate, necessity is not proved.

2.5 Nonce hygiene

The preprocessed pairs (d_{ij}, e_{ij}) are one-time keys in the strictest sense. The paper’s stipulations:

- each pair is used at most once, and the signer *securely deletes* it in the same signing step that returns z_i (Figure 3, step 6);
- an accidentally reused (d_{ij}, e_{ij}) can expose the long-term share s_i ;
- a SA that maliciously replays a commitment pair cannot elicit another response from an honest participant; the replay causes an abort and can therefore deny service;
- the commitment store (if a server) is trusted only for availability and freshness; possession of the public (D_{ij}, E_{ij}) lists grants no signing power.

See §2.8 for RFC 9591’s sharpening of nonce generation (hedged sampling, and an explicit warning that deterministic nonces enable complete key recovery in the multi-party setting).

2.6 The security theorem as stated

The paper gives an EUF-CMA reduction, in the random-oracle model, for an intermediate scheme under the *discrete logarithm problem* in $\mathbb{G}$ — deliberately not to one-more discrete log (the adversary must return the discrete logarithms of $\ell + 1$ challenges after at most ℓ queries to a discrete-log oracle), so that the negative result of Drijvers et al., which rules out security reductions from that assumption for a family of two-round Schnorr multisignatures, does not apply. The proof is for an intermediate scheme, *FROST-Interactive*, in which the binding value is produced by a one-time verifiable random function $\rho_i = a_{ij} + b_{ij} \cdot H_\rho(m, B)$ with committed keys.

Theorem 6.1 (ePrint 2020/852). *If the discrete logarithm problem in $\mathbb{G}$ is (τ', ϵ') -hard, then the FROST-Interactive signature scheme over $\mathbb{G}$ with n signing participants, a threshold of t , and a preprocess batch size of π is $(\tau, n_h, n_p, n_s, \epsilon)$ -secure whenever*

$$\epsilon' \leq \frac{\epsilon^2}{2n_h + (\pi + 1)n_p + 1} \quad \text{and}$$

$$\tau' = 4\tau + (30\pi n_p + (4t - 2)n_s + (n + t - 1)t + 6) \cdot t_{\text{exp}} + O(\pi n_p + n_s + n_h + 1),$$

such that t_{exp} is the time of an exponentiation in $\mathbb{G}$, assuming the number of participants compromised by the adversary is less than the threshold t .

Here n_h, n_p, n_s bound the forger’s random-oracle, preprocess, and signing queries. The architecture is Bellare–Neven generalised forking: the DL challenge ω is embedded as the honest participant’s free-coefficient commitment ϕ_{t0} in a simulated KeyGen, the simulator is run twice on the same tape with oracle answers diverging from index J , and two forgeries yield $\text{dlog}(Y) = (z' - z)/(h'_J - h_J)$. The VRF is what lets the simulator win the programming race: it can compute R before the forger does and program $H_2(R, Y, m)$ with success probability 1.

The step from FROST-Interactive to plain FROST (replace the VRF by $\rho_i = H_1(i, m, B)$) is argued *heuristically* in the paper’s §6.2, not proven — the Equation-1 gap of §2.4. This gap is what the later literature closed, differently and under a different assumption (§2.7).

2.7 The corrected and extended analyses

Three later results matter for how this volume cites FROST’s security.

Crites–Komlo–Maller (ePrint 2021/1375). This work, “How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures” (IACR ePrint 2021/1375, received 2021-10-12, last revised 2022-08-03), introduced the optimisation now called *FROST2*: a single binding factor $d = h_1(vk, lr)$ shared by all signers, replacing the original per-signer $d_i = h_1(vk, lr, i)$ (that original is retroactively *FROST1*) and reducing the signers’ scalar multiplications from linear in the number of signers to constant.

Bellare–Crites–Komlo–Maller–Tessaro–Zhu (CRYPTO 2022). The paper “Better than Advertised Security for Non-interactive Threshold Signatures” — its FROST/BLS part maintained as Bellare–Tessaro–Zhu, ePrint 2022/833, June 2022 — gave the analysis now cited by RFC 9591. The authors define a hierarchy $\text{TS-UF-0} \leq \dots \leq \text{TS-UF-4}$ (and strong variants TS-SUF-2/3/4) grading what counts as a trivial forgery. Their results, proven in the ROM under the *one-more discrete logarithm* (OMDL) assumption, without the AGM, and in the trusted-dealer setting (the DKG is explicitly not modelled):

- FROST1 is TS-SUF-3-secure, and is *not* TS-UF-4-secure;
- FROST2 is TS-SUF-2-secure, and is *not* TS-UF-3-secure — the shared binding factor loses the guarantee that the signer set that committed is the signer set that signed;
- concretely (their Theorem 5.1, verbatim bound), for a TS-SUF-2 adversary A making at most q_s preprocessing and q_h random-oracle queries there is an OMDL adversary B making at most $2q_s + ns$ challenge queries (ns their number of parties) with

$$\text{Adv}_{\text{FROST2}[\mathbb{G}]}^{\text{ts-suf-2}}(A) \leq \sqrt{q \cdot (\text{Adv}_{\mathbb{G}}^{\text{omdl}}(B) + 3q^2/p)}, \quad q = q_s + q_h + 1,$$

and an analogous Theorem 5.4 for FROST1 at TS-SUF-3.

The BCKMTZ analysis for the RFC’s FROST1 signing layer is therefore based on OMDL in the ROM, static corruptions below t , and trusted-dealer keys. It does not establish composition with the DKG shipped by the implementation, and its basis differs from the original paper’s DL reduction for the interactive variant plus a heuristic bridge.

Gouvêa–Komlo (ePrint 2024/436). The preprint “Re-Randomized FROST” (received 2024-03-13) is the design ZIP 312 builds on: FROST unchanged, plus algorithms $\text{GenRand}(\alpha \xleftarrow{\$} \mathbb{Z}_p)$, $\text{RandShare}(\bar{x}_i \leftarrow x_i + \hat{\alpha} \text{ with } \hat{\alpha} = H_r(\alpha, \text{aux}), \text{aux an optional external-protocol transcript contribution})$, $\text{RandPKShare}(\overline{PK}_i = PK_i \cdot g^{\hat{\alpha}})$ and $\text{RandPK}(\overline{PK} = PK \cdot g^{\hat{\alpha}})$. Correctness rests on the $\sum_{i \in S} \lambda_i = 1$ identity, giving $z = r + c(x + \hat{\alpha})$. Its Theorem 1 *claims* unforgeability in the ROM under *algebraic* OMDL (AOMDL). Unlike BCKMTZ’s OMDL game, AOMDL restricts discrete-log-oracle queries to

declared algebraic combinations, so these are not the “same security assumptions” claimed in the preprint’s abstract. The stated bound (its Equation 3) is

$$\text{Adv}_{TS,A}^{\text{sec}} \leq \sqrt{q \cdot \text{Adv}_D^{\text{aomdl}}(\lambda) + 2q^2/p - \text{negl}(\lambda)}, \quad q = q_h + q_s,$$

via the local forking lemma. The intended extraction uses the public randomizer to strip $c^*H_r(\alpha^*, \text{aux}^*)$ from z^* , but the printed proof has material defects recorded in §3.3; the claimed bound is not independently established here. The coordinator chooses the randomizer and is trusted for privacy and unlinkability only, not for unforgeability. The full treatment of this paper belongs to the re-randomisation chapter (§3).

2.8 RFC 9591 against the paper: what the RFC changed

RFC 9591 (IRTF CFRG, Informational, June 2024; Connolly, Komlo, Goldberg, Wood) is the specification implemented by the `frost-core` 3.0.0 workspace this volume cites. Reconciling it against ePrint 2020/852:

- *Two rounds only.* The RFC specifies the two-round protocol; the paper’s single-round-with-preprocessing variant (batch parameter π , commitment server) is declared out of scope.
- *No DKG.* The paper’s Figure-1 KeyGen is dropped; key generation is out of scope, with *trusted-dealer* generation given in Appendix C only. The BCKMTZ analysis the RFC cites matches this framing (it, too, models a trusted dealer).
- *Binding factor inputs widened.* The paper binds $\rho_i = H_1(i, m, B)$. The RFC computes, per participant,

$$\text{binding_factor}_i = H_1(\text{PK_enc} \parallel H_4(\text{msg}) \parallel H_5(\text{encoded commitment list}) \parallel \text{enc}(i)),$$

adding the *group public key* to the hash and pre-hashing the message and commitment list. The per-participant structure means RFC 9591 specifies FROST1: the RFC’s §7.2 rules the single-binding-factor (FROST2) optimisation NOT RECOMMENDED, citing the loss of the started-set-equals-signed-set guarantee (the TS-SUF-3 vs TS-SUF-2 separation of §2.7).

- *Hedged nonces.* The paper samples (d, e) uniformly; the RFC hedges: each nonce is $H_3(r \parallel \text{enc}(sk_i))$ with r a fresh 32-byte random string, so a weak RNG is backstopped by the key share. It warns that fully deterministic derivation enables complete key recovery in the multi-party setting.
- *Five hash roles.* H_1 (binding factor), H_2 (challenge), H_3 (nonce), H_4 (message pre-hash), and H_5 (commitment-list pre-hash) are domain-separated with a per-ciphersuite `contextString`, except that FROST(Ed25519, SHA-512) deliberately leaves H_2 unprefixes for RFC 8032 compatibility. The challenge input order `group_comm_enc` $\parallel$ `PK_enc` $\parallel$ `msg` equals the paper’s $H_2(R, Y, m)$.
- *Roles renamed and channels stated.* The signature aggregator becomes the *Coordinator*; the channel is assumed authenticated. Reliable delivery is also assumed for protocol completion; confidentiality is not required. The Coordinator SHOULD verify the aggregate signature and MAY verify individual shares — the paper’s per-share check $g^{z_i} = R_i \cdot Y_i^{c\lambda_i}$ survives as `verify_signature_share` — with identifiable abort, as in the paper, and no robustness.
- *Ciphersuites.* The RFC ships FROST(Ed25519, SHA-512), FROST(ristretto255, SHA-512) (RECOMMENDED), FROST(Ed448, SHAKE256), FROST(P-256, SHA-256), and FROST(secp256k1, SHA-256); no Zcash suite appears in the RFC itself. The two Zcash suites, FROST(Jubjub, BLAKE2b-512) and FROST(Pallas, BLAKE2b-512), are instead defined by ZIP 312, whose H_2

personalisation `Zcash_RedPallasH` makes the FROST challenge equal RedDSA’s H° — the precise sense in which the RFC protocol plugs into the spend-authorisation verifier of the *Ironwood Guide*, §“Spend authorisation: RedPallas re-randomisation”.

- *Security claim.* The RFC claims EUF-CMA per BCKMTZ, citing both the original paper and BCKMTZ — i.e., the specification’s claim rests on the OMDL/ROM analysis, not the paper’s DL-based theorem for the interactive variant.

Classification: FROST as in RFC 9591 and its Zcash ciphersuites as in ZIP 312 are *specified*; ZIP 312 itself remains Draft status (zips commit 6961098410, 2026-07-09), with its key-generation procedure explicitly out of scope and PedPoP DKG for Zcash deployment *designed-but-unspecified* (implemented in `frost-core`’s `dkg` module but specified in no ZIP or RFC).

3 Re-Randomized FROST for RedPallas

The FROST of §2, specified in RFC 9591 (§2.8), constructs its aggregate relative to a *fixed* group public key PK . Zcash instead supplies a freshly randomised validating key for every Sapling Spend and Orchard Action, and consensus checks the spend-authorisation signature against that per-Action key — a key to which, as §3.1 makes precise, the unchanged FROST shares do not interpolate. This chapter presents the design that closes the gap: the Re-Randomized FROST of Gouvêa and Komlo (IACR ePrint 2024/436), its Zcash specification in ZIP 312, the ciphersuite FROST(Pallas, BLAKE2b-512), the published implementation in the `frost-rerandomized` and `reddsa` crates, and the custody surfaces the construction serves.

A rigour note before the material. RFC 9591 is an Informational RFC recording CFRG consensus, not an Internet Standard. The Gouvêa–Komlo paper is a preprint — received 2024-03-13, no peer-reviewed venue — and ZIP 312 carries Draft status (zips commit 6961098410, 2026-07-09). Following the series rule, every claim beyond the RFC is binned as *specified*, *designed but unspecified*, or an *open problem*; the closing classification is in §3.7.

3.1 The mismatch: consensus verifies only randomised keys

The protocol specification (§4.15, spend authorization signatures) prescribes, for every Sapling Spend and every Orchard Action, the following signer-side flow. The signer chooses a fresh spend-auth randomizer α and:

1. samples $\alpha \leftarrow \text{SpendAuthSig.GenRandom}()$;
2. derives $\text{rsk} = \text{SpendAuthSig.RandomizePrivate}(\alpha, \text{ask})$;
3. derives $\text{rk} = \text{SpendAuthSig.DerivePublic}(\text{rsk})$;
4. proves the Spend or Action statement with α in the auxiliary (secret) input and rk in the primary input;
5. signs: $\text{spendAuthSig} = \text{SpendAuthSig.Sign}_{\text{rsk}}(\text{SigHash})$.

Consensus therefore verifies the signature under $\text{rk} = \text{ak} + [\alpha] G^{\text{SpendAuth}}$. This differs from ak except in the possible identity-randomizer case $\alpha = 0$.

The randomisation algorithms are RedDSA’s, from the protocol specification (§5.4.7):

$$\begin{aligned} \text{RedDSA.RandomizePrivate}(\alpha, \text{sk}) &:= \text{sk} + \alpha \pmod{r_{\mathbb{G}}}, \\ \text{RedDSA.RandomizePublic}(\alpha, \text{vk}) &:= \text{vk} + [\alpha] G, \end{aligned}$$

with identity randomizer 0; `RedDSA.GenRandom` chooses T uniformly among byte sequences of length $(\ell_H + 128)/8$ — 80 bytes at $\ell_H = 512$ — and returns $H^\circ(T)$, where $H^\circ(B) :=$

$\text{LEOS2IP}_{\ell_H}(H(B)) \bmod r_G$. Signing draws its nonce as $r = H^\otimes(T \parallel \text{vk}^* \parallel M)$ for fresh uniform T , sets $R = [r]G$ and $S = (r + H^\otimes(R^* \parallel \text{vk}^* \parallel M) \cdot \text{sk}) \bmod r_G$, and outputs $\sigma = R^* \parallel S^*$. Validation recomputes $c = H^\otimes(R^* \parallel \text{vk}^* \parallel M)$ and accepts iff $R \neq \perp$, $S < r_G$, R^* is canonical (post-ZIP-216), and $[h_G](-[S]G + R + [c]\text{vk}) = 0_G$. RedPallas specialises the template with $\mathbb{G}$ the Pallas group, $\ell_H = 512$, and $H(x) = \text{BLAKE2b-512}(\text{Zcash_RedPallasH}, x)$; $\text{SpendAuthSig}^{\text{Orchard}}$ uses the generator $G^{\text{SpendAuth}} = \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, "G")$ and “is instantiated as RedPallas with key re-randomization”. The security notion the protocol requires of the scheme is SURK-CMA (Strong Unforgeability with Re-randomized Keys), adapted from Fleischhacker et al. (PKC 2016, §3), whose oracle answers queries of the form (m, α) — message and adversary-chosen randomizer together.

The construction and the cited single-party EUF-CMA result are settled in the *Ironwood Guide*: §“Spend authorisation: RedPallas re-randomisation” builds $\text{rsk} := \text{ask} + \alpha$ and $\text{rk} := \text{ak} + [\alpha]G^{\text{SpendAuth}}$, and §“RedPallas unforgeability” proves EUF-CMA under DL in the ROM *including* adversary-chosen α , by key-prefixed forking with $\text{ask} = \text{rsk}^* - \alpha^*$. Per the layering rule we cite those results and do not re-prove them. That proposition is an EUF-CMA statement, not a separate proof of the protocol specification’s strong SURK-CMA notion.

The threshold problem is now visible. A FROST sharing of ask gives shares sk_i with $\sum_{i \in S} \lambda_i \text{sk}_i = \text{ask}$ for every signing set S ; an honest plain-FROST execution constructs a signature under ak . For a nonzero Action randomizer, however, the verifier’s key is rk . The noninteractive construction below shifts every participant by the same effective randomizer. A separately generated Shamir sharing of a randomizer could also shift the group secret, but would be a different, interactive construction.

3.2 Re-Randomized FROST: the construction

The design is due to Gouvêa and Komlo (both Zcash Foundation), “Re-Randomized FROST”, IACR ePrint 2024/436, received 2024-03-13; we work from the full text and page renders of its construction and equations. The transcription below normalises the source-level slips identified after the construction. As in §2 we quote the paper in its own multiplicative notation (g^x for $[x]G$, $PK \cdot g^{\hat{\alpha}}$ for $PK + [\hat{\alpha}]G$); the additive RedPallas form returns with ZIP 312 in §3.4.

Definition 3.1 (Perfectly rerandomizable threshold scheme; ePrint 2024/436, Definition 5). A threshold signature scheme

$$TS = (\text{Setup}, \text{KeyGen}, (\text{Sign}, \text{Sign}'), \text{Combine}, \text{Verify})$$

is *perfectly re-randomizable* if there exist additional PPT algorithms GenRand , RandShare , RandPKShare , and RandPK , and a randomness distribution X , such that the Rand^* algorithms take a randomizer $\alpha \in X$ and an auxiliary string $\text{aux} \in \{0, 1\}^*$.

Remark 3.2 (What “perfectly” specifies). The printed Definition 5 requires only the existence, inputs, and outputs of four algorithms. It states neither a correctness relation nor equality of any distributions. The adjective “perfectly” therefore adds no formal property in that definition. Functional correctness of the concrete scheme comes from Equation (1); no distributional guarantee follows from Definition 5 alone.

The paper’s Remark 1 explains aux : it models an optional contribution from an external protocol transcript, so that a new session changes the re-randomised key even under a repeated α .

Construction 3.3 (Re-Randomized FROST; ePrint 2024/436, Figure 5). The additions to plain

FROST — the paper’s grey-boxed lines — are exactly four algorithms:

$$\begin{aligned} \text{GenRand}() : \alpha &\stackrel{\$}{\leftarrow} \mathbb{Z}_p; \\ \text{RandShare}(x_i, \alpha, \text{aux}) : \hat{\alpha} &\leftarrow H_r(\alpha, \text{aux}); \quad \bar{x}_i \leftarrow x_i + \hat{\alpha}; \\ \text{RandPKShare}(PK_i, \alpha, \text{aux}) : \hat{\alpha} &\leftarrow H_r(\alpha, \text{aux}); \quad \overline{PK}_i \leftarrow PK_i \cdot g^{\hat{\alpha}}; \\ \text{RandPK}(PK, \alpha, \text{aux}) : \hat{\alpha} &\leftarrow H_r(\alpha, \text{aux}); \quad \overline{PK} \leftarrow PK \cdot g^{\hat{\alpha}}. \end{aligned}$$

The FROST core runs unchanged on the shifted inputs. Algorithm $\text{Sign}()$ samples $(r_k, s_k) \stackrel{\$}{\leftarrow} \mathbb{Z}_p^2$ and commits $(R_k, S_k) = (g^{r_k}, g^{s_k})$. Algorithm Sign' , given the randomised share $\bar{x}_k$ and randomised key $\overline{PK}$ in place of x_k and PK , derives each binding factor a_i by H_{non} from the participant identifier, the message, $\overline{PK}$, and the full commitment list $\{(i, R_i, S_i)\}_{i \in S}$, then computes

$$R \leftarrow \prod_{i \in S} R_i \cdot S_i^{a_i}, \quad c \leftarrow H_{\text{sig}}(\overline{PK}, R, m), \quad z_k \leftarrow r_k + s_k a_k + c \lambda_k \bar{x}_k.$$

Algorithm Combine outputs $z \leftarrow \sum_{i \in S} z_i$, $\sigma \leftarrow (R, z)$; $\text{Verify}(\overline{PK}, m, \sigma)$ recomputes the challenge and accepts iff $R \cdot \overline{PK}^c = g^z$.^a

^aThe paper’s own argument order for H_{sig} is inconsistent: Sign' and Combine write $H_{\text{sig}}(PK, R, m)$, while Verify and the reduction’s table write $H_{\text{sig}}(PK, m, R)$ — and both differ from the order RFC 9591 (§4.6) and RedDSA fix, $c = H_2(R^* \parallel PK^* \parallel m)$, which ZIP 312 and the released reddsa crate use. We read the paper’s orderings as a presentational slip with no normative weight; the implemented order is RedDSA’s. The paper also twice refers to “Figure 4” where Figure 5 is meant.

The displayed construction also normalises several evident errors in Figure 5. Its Setup lists only H_{non} and H_{sig} even though every randomisation algorithm calls H_r . In Sign' , its formal argument names the local randomised share $\bar{x}_i$ although the algorithm and its body use signer index k ; we consistently write $\bar{x}_k$. Its loop also prints k as the first input to H_{non} for every a_i ; the iterated index must be i , as above, for the commitment $R = \prod_i R_i S_i^{a_i}$ and response term $s_k a_k$ to use the same per-participant binding factors. The paper’s preliminary Shamir definition has two further typographical slips: its polynomial omits the factor Z from the $a_1 Z$ term and its prose says that t coefficients are sampled while listing $a_1, \dots, a_{t-1}$. Its subsequent evaluation formula is the standard degree- $(t-1)$ sharing used here.

The paper is explicit that this is the whole design: “We do not make any changes to the plain FROST protocol.” Its signing equations are unchanged; the added algorithms supply the shifted share and public-key inputs.

Correctness is one identity. The paper’s Equation (2):

$$z = \sum_{i \in S} r_i + \sum_{i \in S} s_i a_i + c \sum_{i \in S} \lambda_i \bar{x}_i = \sum_{i \in S} r_i + \sum_{i \in S} s_i a_i + c \left(\sum_{i \in S} \lambda_i x_i + \hat{\alpha} \sum_{i \in S} \lambda_i \right) = r + c(x + \hat{\alpha}) = r + c \hat{x}, \quad (1)$$

“because x is Shamir secret shared” and “ $\sum_{i \in S} \lambda_i = 1$ since it is the interpolation of the constant polynomial $f(X) = 1$ ”. The output is a regular re-randomised signature for $\overline{PK} = PK \cdot g^{\hat{\alpha}}$ with $\hat{\alpha} = H_r(\alpha, \text{aux})$.

The identity deserves emphasis, because it is the entire reason the construction is this simple. Every participant adds the *same* $\hat{\alpha}$ to its share — no dealing, no per-participant shares of the randomizer — and the Lagrange weights, summing to one over any interpolating set, carry the common shift through interpolation so that the group secret moves by $\hat{\alpha}$ rather than by $|S| \cdot \hat{\alpha}$. Had the weights summed to anything else, or varied with S in a way the signers could not predict, the shifted shares would interpolate to a key nobody computed.

3.3 Unforgeability: TRUF, AOMDL, and Theorem 1

The game. The paper’s notion (Definition 6, game TRUF in Figure 4) allows adversarial randomizers and static corruption of fewer than t parties. Its signing oracle records only the queried messages Q , and a forgery wins only when $m^* \notin Q$. It is therefore an *existential* notion: a new signature for a previously signed message is not a winning forgery. The protocol specification instead requires SURK-CMA, whose oracle records message–signature pairs and accepts any fresh pair, including a different signature on a previously queried message. The paper therefore uses a weaker freshness condition, and Theorem 1 does not, as printed, establish the protocol specification’s strong notion. The game pseudocode also assigns $\text{RandShare}(x_k, \alpha, \text{aux})$ to $\bar{x}_i$ but later passes $\bar{x}_k$, and its call to Sign' omits the message argument that the construction requires. Those are interface errors in the printed game; the discussion here uses the evident k index and includes m .

There is a second mismatch in the unrandomised branch. The winning predicate also accepts a forgery under PK , but every signing-oracle answer first applies $H_r(\alpha, \text{aux})$; the adversary has no ordinary signing oracle under PK . Contrary to the paper’s prose, setting its outer randomizer α to zero does not produce the identity shift. The construction’s randomizer domain is $\mathbb{Z}_p$ and $H_r(0, \text{aux})$ need not be zero. Thus the separate verification branch does not embed the ordinary EUF-CMA game either.

The assumption. Intended AOMDL is the *algebraic one-more discrete logarithm* game: the adversary receives $\ell + 1$ challenges, may query a discrete-log oracle only on declared linear combinations of environment-generated elements, and should make at most ℓ queries. The algebraic restriction is absent from §2.7’s OMDL. Since it reduces the adversary’s allowed oracle queries, AOMDL hardness is formally a weaker hardness assumption than OMDL hardness, obtained at the cost of an algebraic-adversary restriction; it is neither literally the same assumption nor a stronger hardness assumption. Figure 1 is not a well-defined rendition of that game: its oracle accepts $\{\beta_i\}_{i=1}^\ell$ but its return expression uses β_0 as well, and it never checks that the queried element X equals the declared combination $g^\alpha \prod_i X_i^{\beta_i}$. After the natural repair of supplying coefficients for every challenge, a second defect remains: the game initialises its query counter $q \leftarrow 0$ but never increments it. Its final test $q < \ell + 1$ then imposes no query limit, so an adversary can request every challenge logarithm. We interpret AOMDL by its intended one-more definition below, not by that malformed pseudocode.

The theorem claim. The preprint states the following (Theorem 1 with Equation (3); the radical was checked against the rendered page):

Theorem 1 (ePrint 2024/436). *Rerandomized-FROST is unforgeable in the random oracle model, assuming the AOMDL assumption holds in $\mathbb{G}$, concretely*

$$\text{Adv}_{TS, \mathcal{A}}^{\text{sec}} \leq \sqrt{q \cdot \text{Adv}_D^{\text{aomdl}}(\lambda) + 2q^2/p - \text{negl}(\lambda)}, \quad q = q_h + q_s, \quad (2)$$

with q_h random-oracle and q_s signing queries.

Remark 3.4 (Placement of the negligible term). The printed bound places $-\text{negl}(\lambda)$ *inside* the square root; we transcribe it as printed. The customary form of such forking-lemma bounds — including the BCKMTZ bound quoted in §2.7 — carries additive slack outside the radical, and a negative term inside can make the radicand negative. Neither the theorem statement nor its proof explains how that expression is to be interpreted in that case.

Intended proof shape, and why it is not verified. The reduction $\mathcal{B}$ receives $2q_s + 1$ AOMDL challenges, sets $PK = X_0$, and simulates verification shares in the exponent. Even the displayed public-share simulation is not well formed: its product shadows the output index i and leaves both j

and k unbound. If C is the set of corrupted identifiers and $L_j(i)$ is the Lagrange basis coefficient for node $j \in \{0\} \cup C$ evaluated at the honest identifier i , the required exponent relation is

$$PK_i = PK^{L_0(i)} \prod_{j \in C} (g^{x_j})^{L_j(i)}.$$

The printed expression does not establish that simulated public shares interpolate to PK .

Round-one answers consume challenge pairs. In round two, the response equation

$$z_k = r_k + s_k a_k + c \lambda_k \bar{x}_k$$

requires the reduction to query the logarithm of

$$Z_k = R_k S_k^{a_k} \overline{PK_k}^{c \lambda_k}. \quad (3)$$

The preprint instead prints $R_i S_i^{a_k} \overline{PK_k}^c$: it mixes i and k and omits the Lagrange exponent λ_k . Its claimed “perfect” signing simulation therefore does not follow from the displayed query.

The random-oracle simulation also is not a consistent lazy sampler as printed. Its H_r table stores only the queried pair (α, aux) , not the sampled output, so a repeat query has no associated value to return. Its H_{non} query signature omits m although its lookup and table include m ; when it samples a_j for each other participant, it stores a_k rather than a_j ; and its new-query branch returns the sampled H_{sig} value c rather than the requested binding value a_k . The proof later invokes an unrandomised case $\alpha = \perp$, although neither the construction’s domain $\mathbb{Z}_p$ nor the TRUF oracle defines that sentinel.

The forking step is underspecified too. It says to reprogram $H_{\text{sig}}(PK', R^*, m^*)$ without defining PK' , then asserts a second forgery with the same R^* but potentially different (α, aux) . It does not show that the reprogrammed query is the challenge query for both accepted forgeries when their re-randomised public keys differ. Nor does the proof derive the concrete probability bound in Equation (2); it concludes only with a qualitative “non-negligible probability” invocation of the local forking lemma.

The first fork-extraction identity is algebraically sound if both forgeries share the same commitment R , verify under re-randomised keys, and have $c_0 \neq c_1$. Writing $h_b = H_r(\alpha_b, \text{aux}_b)$, it removes the public shifts and obtains

$$x = \frac{(z_0^* - c_0 h_0) - (z_1^* - c_1 h_1)}{c_0 - c_1}.$$

The subsequent nonce extraction is not sound as printed. If two response shares have the form

$$u_b = r_i + s_i a_b + c_b \lambda_i (x_i + h_b),$$

then for $a_0 \neq a_1$ the corresponding identity is

$$s_i = \frac{u_0 - u_1 - c_0 \lambda_i x_i - c_0 \lambda_i h_0 + c_1 \lambda_i x_i + c_1 \lambda_i h_1}{a_0 - a_1}.$$

The printed numerator gives the two c_1 terms the wrong signs and uses the outer randomizers in place of their H_r images. More seriously, the proof claims that when $c_0 = c_1$, $a_0 = a_1$, and $h_0 \neq h_1$, the reduction can still recover both (r_i, s_i) . After subtracting the known randomizer difference, however, the transcripts give only one linear equation in those two nonce scalars. They are underdetermined.

Together with Figure 1’s missing query-counter increment, these are material gaps, not merely notation. A repair may be possible, but this volume does not supply a complete replacement reduction. It therefore records Theorem 1 as a preprint claim and does not use Equation (2) as a verified assurance statement.

Scope even under a repaired proof. The claimed result is only existential unforgeability in the paper’s weaker game; no cited theorem establishes threshold SURK-CMA. The preprint contains no formal unlinkability definition or theorem. Corruptions are static, and the threshold-scheme game uses centralised key generation; the paper says it “can [be] adapted” to a DKG, but gives no composition proof. Finally, the paper’s §5.1 sketches how a DKG-style contribution round could remove the trusted-for-privacy coordinator (all parties contribute to the randomizer, none learns it) at the cost of extra communication rounds; ZIP 312 does not adopt this. The cost of re-randomisation itself is one fixed-base scalar multiplication per participant and per coordinator; the paper’s Table 1 benchmarks (Pallas suite, Ryzen 9 5900X) put the signing overhead at 83% with 2 signers, 45% at 7, and 8% at 67.

3.4 ZIP 312: the specification

ZIP 312, “FROST for Spend Authorization Multisignatures” (Status Draft, Category Wallet, created 2022-08; owners Gouvêa, Komlo, Connolly; zcash/zips PR #662), specifies Re-Randomized FROST as a delta against RFC 9591. Its target property beyond unforgeability is *unlinkability*, which the ZIP defines as statistical independence of signatures from the signers’ long-term keys: for uniform randomizers and absent metadata leakage, it is “impossible to determine whether two transactions were generated by the same party”.

The signing-flow changes to RFC 9591.

- *Randomizer generation.* A new helper `randomizer_generate(msg, commitment_list)`:
draw `rng_randomizer` $\leftarrow$ `random_bytes(Ns)`, form

`randomizer_input = rng_randomizer ||`
`encode_signing_package(commitment_list, msg),`

and return $H_R(\text{randomizer_input})$. Implementations MAY use another encoding “as long as all values (the message, and the identifier, binding nonce and hiding nonce for each participant) are unambiguously encoded”.

- *Round one* is unchanged.
- *Round two.* The Coordinator generates the randomizer and sends it to each signer “over a confidential and authenticated channel” — plain FROST requires reliability and authentication but not confidentiality — together with the message and the commitment list. Each participant then runs `sign` with $sk_i + \text{randomizer}$ in place of its share and $PK + [\text{randomizer}]G$ in place of the group public key.
- *Aggregation* shifts the group key the same way: $PK \mathrel{+}= [\text{randomizer}]G$.
- *Share verification* shifts both keys: $PK_i \mathrel{+}= [\text{randomizer}]G$ and $PK \mathrel{+}= [\text{randomizer}]G$ in `verify_signature_share`.

The ZIP’s “randomizer” is the paper’s *effective* randomizer $\hat{\alpha}$: per the paper’s §7 implementation notes, `GenRand` and H_r are folded into the single hash call above, and the value distributed to signers is the already-hashed scalar.

The algebra, in RedDSA notation. The ZIP’s Rationale restates the correctness identity of (1) additively, and it is worth displaying because it is the entire consensus-facing content of the design. A valid RedDSA response must satisfy $z = r + c \cdot sk$. Each FROST share contributes

$$z_i = (\text{hiding_nonce}_i + \text{binding_nonce}_i \cdot \rho_i) + \lambda_i \cdot c \cdot sk_i,$$

and under re-randomisation the key terms become $c \cdot (sk + \text{randomizer})$ on the single-signer side and, summed over the coalition,

$$\sum_{i \in S} \lambda_i c(sk_i + \text{randomizer}) = c\left(\sum_{i \in S} \lambda_i sk_i + \text{randomizer} \cdot \sum_{i \in S} \lambda_i\right) = c(sk + \text{randomizer}),$$

since $\sum_i \lambda_i sk_i = sk$ by Shamir and $\sum_i \lambda_i = 1$ (the ZIP proves the latter in its sum-lambda-proof footnote, citing math.SE 1325342). Every participant adds the same randomizer to its share; the weights-sum-to-one identity is exactly why the group secret shifts by α and not by $|S| \cdot \alpha$.

Who learns the randomizer. The threat model is the ZIP’s sharpest departure from RFC 9591. The Coordinator generates the randomizer and is “trusted with the privacy of the transaction (which includes the unlinkability property)” but *not* with unforgeability: “a rogue Coordinator ... should not be able to create signed transactions without the approval of MIN_PARTICIPANTS participants”. All key-share holders receive the randomizer and are likewise trusted with privacy only. The role assignment is natural: the Coordinator fits the transaction builder, who already creates the zero-knowledge proof — and the proof needs α as auxiliary input (step 4 of the flow in §3.1). The confidentiality requirement is not optional hygiene: anyone learning $\hat{\alpha}$ can un-randomise the on-chain key, $ak = rk - [\hat{\alpha}]G$, so the paper advises encrypting the randomizer “so that eavesdroppers can’t break unlinkability”. Non-requirements are stated with equal care: the coordinator-less variant of RFC 9591 §7.5 is unsupported;² share holders can always link the signing operation to the eventual on-chain transaction; key generation is out of scope; network privacy is out of scope.

The message is a hash. The signed message is the SigHash, which by itself does not expose enough transaction semantics for a signer to know what is being authorised. Signers **MUST** therefore check the SigHash against transaction data sent over the same channel, or recompute it themselves; the mechanism is out of the ZIP’s scope. The same authorisation principle applies to any external signer: a signer who cannot see what it signs authorises whatever the builder chose.

Unlinkability, and where it is proven. Neither the paper nor ZIP 312 supplies a formal threshold-unlinkability proof: the paper attempts only an existential-unforgeability proof (§3.3), and the ZIP argues unlinkability in its Rationale by matching distributions — “randomizer_generate generates randomizer uniformly at random as required by RedDSA.GenRandom”, and the signing flow is compatible with RedDSA.RandomizePrivate, RedDSA.RandomizePublic, RedDSA.Sign, and RedDSA.Validate — while its Requirements defer the formal notion to the protocol specification’s SURK-CMA definition. More precisely, reducing a 512-bit random-oracle output modulo the group order is statistically close to uniform, not exactly uniform; the same qualification applies to RedDSA.GenRandom. The distribution-matching argument does connect to a proved statement: the *Ironwood Guide*’s §“Red-Pallas unforgeability” shows honest uniform re-randomisations are distributed as fresh Schnorr keys. What no source supplies is a threshold-specific unlinkability theorem. Nor does the preprint’s weaker existential game establish the threshold SURK-CMA requirement. Bin: the mechanism is *specified* (ZIP 312, Draft); its unlinkability guarantee rests on a distribution argument, and its requested strong-unforgeability guarantee has no matching threshold theorem among the cited sources.

3.5 The ciphersuite FROST(Pallas, BLAKE2b-512)

ZIP 312 defines the ciphersuite in RFC 9591’s template. The group is Pallas with base point $G^{\text{SpendAuth}}$ as in protocol §5.4.7.1, of order p_{Vesta} ; RandomScalar is uniform in $[0, \text{Order} - 1]$; SerializeElement is the point compression $\text{repr}_{\text{Pallas}}(P^*)$ (P^* in series notation) and DeserializeElement is $\text{abst}_{\text{Pallas}}$, erroring on $\perp$ and on the identity element; SerializeScalar is little-endian over 32 bytes and

²ZIP 312’s reference for the variant mislabels it as RFC 9591 “Section 7.3: Removing the Coordinator Role”; the section is 7.5 (§7.3 is “Nonce Reuse Attacks”), and the reference’s URL anchor targets the correct section.

DeserializeScalar rejects values $\geq$ the order. The hash is BLAKE2b-512 with 16-byte personalisation strings and $N_h = 64$; scalar-valued hashes interpret the 64 output bytes as a little-endian integer reduced mod the order. Table 2 lists the oracles.

Oracle	Role	Personalisation
H_1	binding factor	FROST_RedPallasR
H_2	challenge	Zcash_RedPallasH
H_3	nonce	FROST_RedPallasN
H_4	message pre-hash	FROST_RedPallasM
H_5	commitment-list pre-hash	FROST_RedPallasC
H_R	randomizer	FROST_RedPallasA

Table 2: FROST(Pallas, BLAKE2b-512) hash personalisations (ZIP 312). The challenge hash reuses RedPallas’s own personalisation; the FROST additions use the FROST_ prefix.

The single load-bearing row is H_2 : ZIP 312 states it is “equivalent to $H^{\otimes}(m)$ ” of RedPallas, so the FROST challenge is the on-chain challenge, and signature verification for the suite is RedPallas validation itself. The ZIP’s compatibility rationale completes the argument that a FROST aggregate is a plain RedPallas signature: the (R, S) output split matches RedDSA.Validate’s parsing; FROST and Zcash use the same challenge input order (R , public key, message); and the fact that r and R are FROST-generated rather than produced by RedDSA.Sign’s nonce derivation is irrelevant to a verifier, which only parses R and checks the validation equation. A parallel suite FROST(Jubjub, BLAKE2b-512) — personalisations FROST_RedJubjub $\{R, N, M, C, A\}$ and Zcash_RedJubjubH — serves Sapling spend authorisation. ZIP 312 defines that suite separately; this volume develops only the Pallas suite.

3.6 Published library implementation: frost-rerandomized and reddsa

Claims in this subsection cite ZcashFoundation/frost at commit 2016e44 (2026-04-23, workspace version 3.0.0) and ZcashFoundation/reddsa at commit 3792daa (2026-05-22, crate version 0.5.2), rechecked against current heads on 2026-09-05. The pins are published releases — tag frost-core/v3.0.0 (crates.io 2026-04-23) and reddsa 0.5.2 — so main-at-commit and the released crates coincide. ZIP 312 names reddsa as the reference implementation of both ciphersuites. A current-head check on 2026-09-05 found frost 0966bd1 and reddsa 63c6ac0; their post-release changes update the Rust edition, minimum compiler, dependencies, benchmarks, and CI, not the re-randomized protocol derived below.

The generic layer. Crate frost-rerandomized (frost-rerandomized/src/lib.rs) implements Construction 3.3 over any frost-core ciphersuite. Trait impl Randomize<KeyPackage> computes the randomised signing share as signing_share + randomizer and the randomised verifying share by adding the randomizer_element; struct RandomizedParams carries the randomizer (“also called alpha” in its documentation), randomizer_element = [randomizer] G , and randomized_verifying_key = $PK + \text{randomizer_element}$. The current flow differs instructively from a literal reading of the ZIP: the Coordinator sends a randomizer *seed*, not the randomizer, and each participant recomputes the randomizer from that seed and the full round-one commitment map in its signing package. The derivation, its divergence from the draft text — the message is no longer a hash input — and the reconciliation in flight in zips PR #895 are treated once, in §4.2.

The RedPallas suite. Module src/frost/redpallas.rs in reddsa implements the frost-core Ciphersuite trait for PallasBlake2b512 (contextual ID “FROST(Pallas, BLAKE2b-512)”). Its generator() is orchard::SpendAuth::basepoint(), whose constant bytes are reproducible as pallas::Point::hash_to_curve("z.cash:Orchard")(b"G") — the protocol’s $G^{\text{SpendAuth}}$. Oracles $H_1/H_3/H_4/H_5$ carry the FROST_RedPallas $\{R, N, M, C\}$ personalisations of Table 2; H_2

goes through the crate’s HStar default, `Zcash_RedPallasH`; the `RandomizedCiphersuite::hash_randomizer` impl uses `FROST_RedPallasA` (the ZIP’s H_R); scalar reduction is the 64-byte-wide `from_bytes_wide` (`src/hash.rs`), whose Pallas implementation delegates to pasta’s `from_uniform_bytes` (`src/orchard.rs`). Beyond ZIP 312, the crate defines two further personalisations — `HDKG = FROST_RedPallasD` and `HID = FROST_RedPallasI`, the DKG and identifier hashes — for key generation, which the ZIP does not specify at all.

Even- Y normalisation. One deployment detail appears in neither the paper nor the ZIP 312 body: the `reddsa` hooks `post_generate/post_dkg` normalise fresh key material to the even- Y representative the protocol specification requires of `ak`; the full mechanism is §4.1. The *randomised* key `rk` need not satisfy the even- Y condition — only `ak` does.

The RedPallas re-randomised API. Module `src/frost/redpallas/rerandomized.rs` re-exports the re-randomised flow for the suite — `sign_with_randomizer_seed()`, `aggregate()`, and `aggregate_custom()` — together with the type aliases `Randomizer` and `RandomizedParams` over `PallasBlake2b512`. The module README’s worked example runs the full trusted-dealer plus re-randomised signing flow and ends by verifying the aggregate under the `randomized_verifying_key()` of its `RandomizedParams` — the executable form of this chapter’s claim that a FROST aggregate is a plain RedPallas signature under `rk`.

3.7 Custody surfaces and classification

(a) Spend-authorisation custody. The primary surface: a t -of- n sharing of `ask` signing Orchard Actions (and, via the Jubjub twin suite, Sapling Spends). FROST replaces step 5 of the signer flow in §3.1; steps 1–4 move to the Coordinator-as-builder, who draws the randomizer, proves the Action statement with α as auxiliary input, and publishes `rk` — the circuit and consensus rules are untouched, per the *Ironwood Guide*, §“Spend authorisation: RedPallas re-randomisation”. Bin: *specified* (ZIP 312, Draft), with key generation explicitly out of the ZIP’s scope; DKG-based key generation for Zcash remains *designed but unspecified* (implemented in `frost-core`, specified nowhere), as §2.8 already concluded for the base protocol. Related anticipations elsewhere in the ZIP corpus: ZIP 316 requires FROST unified viewing key material to follow ZIPs 312 and 2005, and ZIP 271 anticipates lockbox disbursement to “a FROST address”, noting (as of May 2025) that ZIP 312 does not specify key generation.

(b) ZIP 2005 quantum spending key custody. ZIP 2005’s “Usage with FROST” section constrains threshold deployments of its quantum-recovery hierarchy: the key `ak` is derived jointly, by FROST DKG, from participants’ shares of `ask`, and the participants privately agree on a value `sk` from which `nk`, `qsk`, `qk`, and `rivkext` derive. The full ceremony, its derivations, and its asymmetric threat model — every participant holds `qsk`, so the threshold property does not survive an adversary that can also solve Pallas discrete logarithms — are developed once, in §4.7. Bin: the FROST-facing requirements are *specified* (ZIP 2005’s requirements list calls for full compatibility with ZIP 312 FROST, hardware wallets, and their combination); the combined DKG-plus-shared-`sk` key generation is *designed but unspecified* — no source specifies the protocol by which participants “privately agree” on `sk`.

(c) Crosslink finalizer keys. The *Crosslink Guide*, §“Finalizer keys”, bins threshold custody of finalizer keys as an *open problem*, and we adopt that bin. One precision for this volume: the applicable protocol there is *plain* FROST — the finalizer design has no key re-randomisation, so the natural suite is RFC 9591’s FROST(Ed25519, SHA-512), not either ZIP 312 suite. The prototype state and this observation are developed once, in §4.8.

(d) PCZT signing. The multiparty transaction flow that would host the Coordinator role is the PCZT of the *Wallet Guide*, §“Partially created transactions (PCZT)”: ZIP 374 (Partially Created Zcash Transaction Format, Draft, owner Jack Grigg) names threshold signing (“for example FROST”, per RFC 9591) in its Motivation among the multiparty use cases its Signer role must accommodate.

Classification. In the series’ three bins: base FROST and its five ciphersuites are *specified* in RFC 9591, an Informational RFC recording CFRG consensus (§2.8). Re-Randomized FROST, its threat model, and both Zcash ciphersuites are *specified* by ZIP 312 at Draft status. The preprint’s claimed existential-unforgeability proof has the gaps in §3.3, no cited theorem establishes threshold SURK-CMA, and unlinkability rests on a distribution argument rather than a theorem. Key generation — FROST DKG for Zcash, and ZIP 2005’s shared-sk agreement — is *designed but unspecified*; threshold custody of Crosslink finalizer keys is an *open problem*. Implementation-behaviour claims (seed-based randomizer derivation, even- Y normalisation, the extra DKG personalisations) are implementation facts of the pinned commits, cited as such.

4 Deployment and open questions

The preceding sections dealt in designs and proofs; this one deals in what ships. We pin the exact code paths available to a prospective Zcash threshold-custody deployment, reconcile them against the draft ZIP text where the two have diverged, then walk the three custody surfaces this volume targets — spend authorisation through PCZT, the ZIP-2005 quantum spending key, and Crosslink finalizer keys — and close by sorting every artefact into the series’ three bins (*specified*, *designed-but-unspecified*, *open problem*), each with the concrete trigger on which its classification must be revisited.

4.1 The shipped library stack

Two Zcash Foundation repositories carry the implementation. The `frost` workspace (commit 2016e44ba4, 2026-04-23, tagged `frost-core/v3.0.0`; workspace version 3.0.0, MSRV 1.81) provides the generic `frost-core`, the five RFC 9591 ciphersuite crates `frost-ristretto255`, `-ed25519`, `-ed448`, `-p256`, and `-secp256k1`, the separate `frost-secp256k1-tr` Taproot suite, and the generic re-randomisation layer `frost-rerandomized`. Beyond RFC 9591’s two-round protocol the workspace implements trusted-dealer key generation (the RFC’s appendix), a variant of the original paper’s Pedersen-with-knowledge-proof DKG that omits its context string (ePrint 2020/852, §2.2 — not in the RFC), Repairable-Threshold-Scheme share recovery (ePrint 2017/1155), share refresh, and Re-Randomized FROST (ePrint 2024/436, §3). The README declares the crates “considered stable and feature complete”, with SemVer guarantees.

The `frost-rerandomized` crate is `no_std`; its `Randomize` implementations add the randomizer group element to every verifying share of a `PublicKeyPackage` and the randomizer scalar to the signing share of a `KeyPackage`, whose `randomize` documentation warns: “You MUST NOT reuse the randomized key package for more than one signing.” Its README notes that “the main ciphersuite crates do not re-expose the rerandomization functions... The exceptions are the Zcash ciphersuites in `reddsa` which do expose the randomized functionality” — the Zcash ciphersuite modules are the shipped top-level suites that re-export those generic functions.

The `reddsa` crate (commit 3792daa95e, 2026-05-22, version 0.5.2) supplies those ciphersuites: its feature `frost = ["frost-rerandomized", "alloc"]` pulls `frost-rerandomized` 3.0.0, and its modules `reddsa::frost::redpallas` and `reddsa::frost::redjubjub` implement the two ZIP-312 ciphersuites (PallasBlake2b512, JubjubBlake2b512), each with a `keys` module (trusted-dealer, DKG, repairable) and a `rerandomized` submodule re-exporting `sign_with_randomizer_seed`, `aggregate`, `aggregate_custom`, `Randomizer`, and `RandomizedParams`. Function `hash_randomizer` instantiates ZIP 312’s H_R as $H^\circledast$ (the code’s `HStar`) under the personalisation `FROST_RedPallasA`

(respectively `FROST_RedJubjubA`), matching §3.5. The crate’s README calls this feature experimental because ZIP 312 is still Draft and states that neither its SemVer nor its MSRV guarantee applies to the feature.

The `RedPallas` module additionally enforces the even- y requirement that ZIP 2005’s amended protocol-spec §4.2.3 places on `ak` (§4.7): `trait keys::EvenY` negates the group verifying key, every verifying share, the signing shares, and the VSS commitment coefficients whenever the serialised key has $\tilde{y} = 1$ (the code’s evenness test is `serialized[31] & 0x80 == 0`), and the `post_generate` and `post_dkg` ciphersuite hooks call `into_even_y` on fresh key material, so trusted-dealer and DKG outputs alike land on the even- y representative. Negation commutes with Shamir sharing because it is linear. The `RedJubjub` module carries no such machinery; the constraint is Pallas/Orchard-specific.

4.2 The randomizer flow: draft ZIP against shipped crate

ZIP 312 (“FROST for Spend Authorization Multisignatures”; owners Gouvea, Komlo, Connolly; Status Draft; created 2022-08) specifies, at zips commit 6961098410, the randomizer generation

```
signing_package_enc = encode_signing_package(commitment_list,msg),
randomizer = HR(random_bytes(Ns) || signing_package_enc),
```

with H_R for `FROST(Pallas, BLAKE2b-512)` being `BLAKE2b-512` under the personalisation `FROST_RedPallasA`, output reduced to a Pallas scalar (an element of $\mathbb{F}_{p_{\text{Vesta}}}$); H_2 is `Zcash_RedPallasH`, i.e. exactly `RedPallas`’s $H^\circledast$ (§3.5). For Jubjub the personalisations are `FROST_RedJubjub{R,N,M,C,A}` and `Zcash_RedJubjubH`, with base point G^{Sapling} . Round 2 then proceeds as plain FROST with three substitutions,

$$\begin{aligned} sk_i &\leftarrow sk_i + \text{randomizer}, & PK_i &\leftarrow PK_i + \text{ScalarBaseMult}(\text{randomizer}), \\ \text{group_public_key} &\leftarrow \text{group_public_key} + \text{ScalarBaseMult}(\text{randomizer}), \end{aligned}$$

the second applying in share verification. The Coordinator sends the randomizer over a “confidential and authenticated channel” — adding confidentiality to plain FROST’s reliable, authenticated-channel assumption, because the randomizer links the ceremony to the on-chain rk .

The released crate differs from this draft text. Crate `frost-rerandomized 3.0.0` deprecates `Randomizer::new(rng, signing_package)` — the earlier, message-inclusive API. Even that API is not byte-for-byte the draft flow: it samples and serialises a field scalar before hashing, whereas the ZIP starts with N_s unconstrained random bytes. The recommended API is instead `Randomizer::new_from_commitments`: the Coordinator draws an N_s -byte `randomizer_seed`, sends the *seed*, and each participant regenerates

```
randomizer = hash_randomizer(seed || encode_group_commitments(commitments))
```

locally via `regenerate_from_seed_and_commitments`. The message is no longer an input. The crate documentation says that hashing the complete commitment map hedges the Coordinator’s RNG because the map includes signer commitments. This is its implementation rationale, not a property proved by the cited RFC, ZIP, or preprint; in particular, the Coordinator receives the commitments before generating the seed and remains trusted for unlinkability. In the paper’s terms (ePrint 2024/436, Definition 5 and Remark 1), the commitment encoding plays the role of the optional contributory string *aux*, which models “the transcript of some external protocol”; the paper’s own §7 implementation notes describe the older design, with the message still included, in the same terms.

The gap between draft and crate is being closed in zips PR #895 (“[ZIP 312] Specify key generation, change randomizer handling”; opened August 2024, 21 commits, still open and mergeable as of 2026-09-05, with its latest review comments dated 2026-08-28). It adds a key-generation specification, revises the randomizer handling (the randomizer “couldn’t possibly be generated using the message as an input”), integrates the ZIP-2005 qsk material, renames the scheme consistently to “Re-Randomized FROST”, and adds Daira-Emma Hopwood as a ZIP owner. Its commit history also retires

the draft’s serialisation reference — the ZF FROST book’s moving-target serialisation-format page — in favour of RFC 9591’s serialisation function. Until it merges, this volume presents the crate flow — message omission included — as the observed fact of the pinned commit and the ZIP text as a divergent Draft specification. No conformance conclusion follows while that text remains in flight.

The trust geometry is worth stating exactly. The paper’s §5.1 holds that the central randomizer-choosing party is “trusted with preserving privacy of the scheme” but “not trusted with security”, and that removing it would require a DKG-style contributory sub-protocol at extra rounds and complexity. ZIP 312’s threat model matches: the Coordinator is trusted with transaction privacy and unlinkability, while the requirements say it must not be able to forge without MIN_PARTICIPANTS approvals; every key-share holder is trusted with transaction privacy. Non-requirements are equally explicit: ZIP 312 does not remove the Coordinator role, does not (at this commit) provide key generation, and places network privacy out of scope.

4.3 PCZT as the integration surface

ZIP 374’s Motivation names “multiparty transactions — threshold signing (for example FROST [RFC 9591])” among the problems the PCZT format solves; the *Wallet Guide*, §“Partially created transactions (PCZT)”, documents the format and its role graph. The insertion point for FROST is the Signer role (*Wallet Guide*, §“Signing”): to sign an Orchard spend the action’s re-randomisation scalar α must be present, the local-key path computes $rsk := ask + \alpha$ — the re-randomisation of the *Ironwood Guide*, §“Spend authorisation: RedPallas re-randomisation” — and checks $VerificationKey(rsk) = rk$, and the *external* path, `apply_orchard_signature`, accepts a signature if and only if it verifies under the action’s rk over the shielded sighash (`InvalidExternalSignature` otherwise). A FROST-aggregated RedPallas signature is exactly such an external signature; for it to verify, the effective FROST randomizer must equal the PCZT action’s α , so that the re-randomised group key `RandPK` produces equals that action’s rk .

PCZT v2 has since shipped in `pczt` 0.8.0. Current `pczt` 0.9.3 carries separate Orchard and Ironwood bundles and tags an externally produced `SpendAuthSignature` with its value pool and action index; `apply_orchard_spend_auth_signature` dispatches to the corresponding bundle and verifies under that action’s rk . The invariant in the preceding paragraph — one shared shielded sighash and the action’s own α — is unchanged for v6 Ironwood spends (`librustzcash` 91f448b5, `pczt/src/roles/signer/mod.rs`).

The end-to-end flow has been demonstrated (ZF FROST book, `zcash/devtool-demo.md`):

1. `zcash-devtool pczt create` produces the PCZT;
2. the `zcash-sign` tool (from `ZcashFoundation/frost-zcash-demo`, now `frost-tools`) prints a SIGHASH and a Randomizer from the transaction plan;
3. `frost-client` runs the FROST protocol over `frostd` with that SIGHASH as the message and that randomizer;
4. `zcash-sign sign` writes the aggregated signature into `frost_pczt.signed`;
5. `zcash-devtool pczt combine` merges the signed and proven PCZTs;
6. `zcash-devtool pczt send` broadcasts.

The current `frost-tools` head (06c0dbdb, 2026-08-12) does not exercise the version-3 seed flow described in §4.2. Its lockfile selects `frost-core` 2.2.0, `frost-rerandomized` 2.1.0, and an older `reddsa` 0.5.1 revision. For PCZT, the Coordinator accepts each Action’s already-generated scalar randomizer and sends that value to participants through the legacy `raw-randomizer` API. This is a dated implementation boundary: the demo validates the PCZT external-signature path, but it is not evidence for integration of the pinned version-3 library stack or its seed flow.

That distinction creates an ordering requirement for any version-3 integration. The recommended seed API derives the effective randomizer from round-one commitments, so it cannot in general be made equal to an α already fixed inside a PCZT Action. The transaction builder must instead use the derived randomizer as that Action's α before constructing its proof and rk , or use a separately justified compatibility path. Neither the legacy demo nor a specification cited here demonstrates that reordered end-to-end flow.

Here `zcash-devtool` builds on `zcash_keys` with the features "orchard", "unstable", and "unstable-frost". The `unstable-frost` feature (`= ["orchard"]`) exposes the constructor `UnifiedFullViewingKey::from_orchard_fvk` — a UFVK from an Orchard FVK alone, since a FROST wallet has no unified spending key — and the `zcash_keys` changelog (0.3.0, 2024-08-19) frames the flag as existing “to temporarily expose API features that are needed specifically when creating FROST threshold signatures”, to be removed “once key derivation for FROST has been fully specified and implemented”. Repository `librustzcash` itself, that is, classifies FROST key derivation as not fully specified.

One check remains outside every specification: ZIP 312 requires that signers “MUST check that the given SIGHASH matches the data sent from the Coordinator, or compute the SIGHASH themselves”, and explicitly leaves the mechanism out of scope. In the demonstrated flow the SIGHASH travels from `zcash-sign` to the participants through `frost-client`; by what mechanism a share-holding participant gains independent confidence in it is unspecified.

4.4 Wallet-integration constraints

The ZF FROST book's technical-details chapter records the constraints a FROST-capable Zcash wallet inherits. The documented Zcash integration cannot authorise transparent inputs, which require ECDSA; the book accordingly suggests supporting Orchard only (“Orchard is simpler to handle”). That is historical integration advice rather than a current-pool limit: the page predates the Ironwood pool. With a DKG-generated ak , the remaining Orchard key components derive from ak plus either an sk used only for nk and $rivk$, or independently generated nk and $rivk$. Under this key-generation flow, a seed phrase alone does not restore the wallet: recovery also needs the key share, all verifying shares, the participant identifiers, and the FVK components. The book stores these in participant configuration files; the library also provides the Repairable Threshold Scheme (§4.1).

The Foundation's status report (“The State of FROST for Zcash”, 2025-05-20) named wallet integration the missing piece (“The missing piece is for wallets to integrate FROST”): at publication time it reported no mainstream wallet integration, while “technical-inclined users can use FROST with Zcash today using our `frost-client` command line tool”. The same post lists the “lack of a standardized key generation specification for FROST” as the interoperability limiter, flags metadata protection (e.g. Tor) as needing work, and offers ZF as a candidate operator of a production `frostd`.

4.5 Transport: `frostd` and `frost-client`

The reference transport is `frostd`, a JSON-over-HTTPS session server: clients authenticate with key pairs, the Coordinator creates sessions naming the participants' public keys, and all FROST messages are end-to-end encrypted. Thus the server need not be trusted with message contents or signing authority, although it remains an availability dependency. The CLI `frost-client` drives it; both live in `ZcashFoundation/frost-tools` (renamed from `frost-zcash-demo`). Least Authority, as Zcash Ecosystem Security Lead, audited `frost-client` and `frostd` (announcement 2025-05-01) with no high-severity findings, all findings addressed on `main`. No ZIP specifies this transport; ZIP 312 places network privacy out of scope (§4.2). The tools repository labels the command-line programs as demos and says they should not be used in production. The `frost-client` README also warns that it stores secrets unencrypted in its configuration file and recommends protecting that file or building a separate tool from the client as a base.

4.6 Audit coverage, precisely

Two cryptographic-library audits are relevant, and neither covers the code path Zcash custody would run. NCC Group audited the v0.6.0 release (commit 5fa17ed) of `frost-core`, `frost-ed25519`, `frost-ed448`, `frost-p256`, `frost-secp256k1`, and `frost-ristretto255` — including trusted-dealer and DKG key generation and FROST signing — with all findings addressed and reviewed by NCC (public report 2023-10-23). The repository README is explicit: “This does not include `frost-secp256k1-tr` and rerandomized FROST.” Earlier, the 2021 “Audit of FROST implementation” (JP Aumasson and Adrien Hamelink, Taurus; Omer Shlomovits, ZenGo; dated 2021-03-23) covered the pre-rewrite implementation in the RedJubjub repository’s `src/frost.rs` (404 source lines by the report’s count, at commit 2ebc08f, 2021-02-25) — two-round FROST with a trusted dealer only, RedJubjub only — concluding “We did not find any major security issue.” The report also records that the implementation omitted the paper’s secure deletion of nonces and commitments after signing.

The composition, therefore: `frost-rerandomized` plus the `reddsa` RedPallas/RedJubjub ciphersuites — the exact stack of §4.1 — is not covered by any completed third-party audit identified in the repository and audit announcements rechecked 2026-09-05. Of those audits, only the 2021 review touched a Zcash FROST ciphersuite, and it predates the `frost-core` rewrite. The separate Least Authority audit covers the transport tools, not this cryptographic composition.

4.7 Custody of the ZIP-2005 quantum spending key

ZIP 2005 (“Ironwood Quantum Recoverability”; owners Daira-Emma Hopwood and Jack Grigg; Status Proposed, Category Consensus; activation at the NU6.3 activation height) is the second custody surface. Its “Usage with FROST” section derives `ak` jointly, from the participants’ shares of `ask`, via FROST DKG. It then says that participants “MUST privately agree on a value `sk`” and use it with `use_qsk = true` to derive the remaining material. The protocol must give every participant the same extracted validating-key value `ak`, `nk`, and `rivk_ext`; the specified derivations then imply equal `qsk` and `qk` under the ZIP’s stated collision-resistance assumptions. In this series, $ak = [ask]G^{\text{SpendAuth}}$ denotes the Pallas point. ZIP 2005 writes $ak^{\mathbb{P}}$ for that point and $ak := \text{Extract}_{\mathbb{P}}(ak^{\mathbb{P}})$ for its extracted field element. With that point/field-element distinction explicit, its derivations are

$$\begin{aligned} H^{\text{qsk}}(sk) &= \text{truncate}_{32}(\text{PRFexpand}_{sk}([0x0C])), \\ H^{\text{qk}}(qsk) &= \text{BLAKE3.derive_key}(\text{"Zcash ZIP 2005 qk-derivation v1"}, qsk, 32), \\ H_{qk}^{\text{rivk_ext}}(ak, nk) &= \text{ToScalar}^{\text{Orchard}}(\text{PRFexpand}_{qk}([0x0D] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))). \end{aligned}$$

The displayed key derivations use the `PRFexpand` domain-separator bytes `0x0C` and `0x0D`; ZIP 2005 also reserves `0x0A` and `0x0B` for note-related uses. In the `use_qsk = true` branch of the amended protocol-spec §4.2.3, the wallet obtains the point $ak^{\mathbb{P}}$ “by any suitably secure method that ensures the last bit of $\text{repr}_{\mathbb{P}}(ak^{\mathbb{P}})$ is 0” — the requirement the `reddsa` EvenY machinery of §4.1 enforces — and sets the ZIP’s field element $ak = \text{Extract}_{\mathbb{P}}(ak^{\mathbb{P}})$, $qsk = H^{\text{qsk}}(sk)$, $qk = H^{\text{qk}}(qsk)$, and $rivk = H_{qk}^{\text{rivk_ext}}(ak, nk)$. The ZIP’s list of admissible generation methods for $ak^{\mathbb{P}}$ explicitly includes “FROST distributed key generation [zip-0312-key-generation] in which the corresponding spend-authorization material is *t*-of-*n* shared among the participants”.

The threat model shifts accordingly. The host wallet in a multi-signer FROST setup is the ZIP-312 Coordinator; the ZIP gives it `nk`, the extracted validating-key value `ak`, and `qk`. With FROST and hardware wallets, spend authorisation breaks only by (1) possession of `qsk` *plus* finding discrete logarithms on Pallas — and “the adversary is an authorized FROST signer — they hold a copy of `qsk` by construction” is the ZIP’s variant 1.a — or (2) breaking RedDSA, FROST, the DKG, or the proving system. Each participant MUST treat `qsk` as a secret held at least as securely and reliably as `ask`: it is not needed until the Recovery Protocol, but losing it can lose funds once the current Orchard protocol is disabled. A quantum adversary holding `qsk` — which every participant holds — and able to

solve discrete logarithms on Pallas may steal funds, so the ZIP RECOMMENDS that once a fully post-quantum threshold multisignature protocol exists, all FROST-held funds migrate to its pool. FROST’s key advantage — signing with shares without reconstructing ask during signing — would be retained in the outlined Recovery Protocol, which would still require a RedDSA signature.

One tension is unresolved. FROST DKG exists precisely to avoid a commonly-known secret, yet the ceremony above requires the participants to “privately agree on a value sk ” from which nk and qsk derive — and neither ZIP 2005 nor the crates give a protocol for that agreement. ZIP 2005’s Deployment section points at PR #895 (“There is no significant existing deployment of FROST, so we can write this into ZIP 312 from the start”), so the resolution is expected there: as of 2026-09-05 the PR’s commit history drafts a contributory sk -generation section (2026-03-05), still unmerged, so the shared- sk agreement stays *designed but unspecified* with #895’s merge as its trigger.

4.8 Crosslink finalizer keys

The third surface is prospective only. The *Crosslink Guide*, §“Finalizer keys”, records that the prototype uses single ed25519 keys per finalizer (ed25519-zebra), with 76-byte votes signed by appending a 64-byte ed25519 signature over those 76 bytes, and that no FROST or threshold-signing design for finalizer keys is sourced anywhere in the Crosslink corpus (negative search over tfl-book, crosslink-protocol-guide, crosslink-spec, and zebra-crosslink, dated 2026-07-15 in that guide); key rotation, revocation, threshold custody, and initial-roster formation are classified open problems in every design source.

We record one observation of our own, clearly marked as such: finalizer votes are plain ed25519, with no key re-randomisation, so the fitting ciphersuite would be RFC 9591’s FROST(Ed25519, SHA-512) via the `frost-ed25519` crate, whose v0.6.0 predecessor was covered by the NCC audit, with no re-randomisation layer — finalizer keys are deliberately long-lived roster identities, not unlinkable one-time keys, so the entire apparatus of §3 is unnecessary there. No published design says this; it is this volume’s inference from the vote format and the audit table.

4.9 Classification and revisit triggers

Per the series’ three-way scheme:

- *Specified.* FROST as in RFC 9591 (§2.8); Re-Randomized FROST for spend-authorisation custody, per ZIP 312 at Draft status — PR #895’s in-flight randomizer and key-generation changes are its named revisit trigger and do not move the bin (§4.2); the `frost-core/frost-rerandomized/reddsa` API surface at the pinned versions (§4.1); the PCZT Signer external-signature path (§4.3).
- *Designed-but-unspecified.* FROST key generation for Zcash (out of scope at the pinned ZIP-312 commit; the crates implement the PedPoP DKG of ePrint 2020/852, but no ZIP specifies it); the ZIP-2005 `use_qsk` FROST key-generation constraints (ZIP Proposed, referencing the unmerged #895; the sk -agreement step has no protocol, §4.7); the transport (`frostd` exists and is audited, but no ZIP covers it, §4.5).
- *Open problem.* Crosslink finalizer threshold custody (§4.8); post-quantum threshold signing for the qsk -era migration (§4.7); hardware-wallet FROST share custody — ZIP 2005’s threat model assumes hardware-wallet FROST signers, but none was identified in the device integrations checked for this audit.

Two former revisit triggers have fired without changing these bins. NU6.3 is active, but `zcash_keys` still gates `from_orchard_fvk` behind `unstable-frost` pending a specified FROST derivation. PCZT v2 is released, and the current signer preserves the α and external-signature surface for Ironwood as described in §4.3. The remaining classifications expire on the following identifiable events, in rough order of expected impact:

- *PR #895 merges.* The randomizer flow, the key-generation specification, the sk-agreement protocol, and the ZIP-312/ZIP-2005 integration all re-derive from the merged text; the serialisation reference it pins replaces the ZF-FROST-book moving target (§4.2).
- *An audit of the re-randomised stack is announced.* The “no completed third-party audit” claim of §4.6 must be rechecked immediately before any printing in any case.
- *A finalizer key-management design is published.* The Crosslink bin moves, and the frost-ed25519 observation is either confirmed or discarded (§4.8).
- *A post-quantum threshold multisignature protocol exists.* ZIP 2005’s RECOMMENDED migration of FROST-held funds becomes actionable (§4.7).